

# Project 3: Flask Auth Microservice CI/CD Demo

## Overview

This project demonstrates deploying a Python Flask application via Jenkins. The key learning objective is **Handling environment variables and secrets during deployment**.

## Prerequisites

- Docker, Trivy, Python 3, pip running on Jenkins server.
- A Jenkins **Secret text** credential created with ID: ``demo-auth-secret``.

## Step 1: Network Setup

Create a shared Docker network for your infrastructure and deployments:

```
docker network create jenkins-net
```

## Step 2: Infrastructure Setup

Run SonarQube and Jenkins on the ``jenkins-net`` network:

```
# Run SonarQube
```

```
docker run -d --name sonarqube -p 9000:9000 --network jenkins-net  
sonarqube:lts-community
```

```
# Run Jenkins
```

```
docker run -d --name jenkins -p 8080:8080 -p 50000:50000 --network  
jenkins-net -v /var/run/docker.sock:/var/run/docker.sock -v  
jenkins_home:/var/jenkins_home jenkins/jenkins:lts
```

## Step 3: Jenkins Configuration

1. Install Plugins: Docker Pipeline, SonarQube Scanner, Pipeline, Credentials Binding.
2. Configure SonarQube Server in Manage Jenkins -> System.
3. **Crucial Step:** Go to Manage Jenkins -> Credentials -> System -> Global credentials.
  - Click "Add Credentials".
  - Kind: "Secret text".
  - Secret: Type any secure password (e.g., "MySuperSecretKey123!").
  - ID: ``demo-auth-secret``.
  - Description: Demo secret for Flask.

## Step 4: Pipeline Execution

1. Create a new Pipeline job.
2. Use the provided `Jenkinsfile`.
3. Run the build. Observe how the credentials block securely masks the variable in the console output.

## Step 5: Verification

Access the deployed Flask API:

- URL: `http://localhost:5000/auth`

- Expected Output: `{"message":"Secret successfully injected via CI/CD!", "secret_length":20, "status":"Auth Active"}`